

The $q(x)$ update given $q(f)$: two evaluation paths

Picking up where `efgp_estep_succinct.tex` leaves off: the $q(f)$ step is done, so we hold the posterior-mean drift $\bar{f}_r(x) = \phi(x)^* \mu_r$ fixed on the spectral grid $\{\xi_k\}_{k=1}^M$ (weights D_k , centre x_c), and take one SING natural-gradient step on $q(x)$. Diffusion is isotropic, $\Sigma = \sigma^2 I$.

1 What we compute (common to both paths)

1. By exponential-family duality the natural gradient is $\partial \mathcal{L} / \partial \mu$, $\mu = (\mathbb{E}[x_i], \mathbb{E}[x_i x_i^\top], \mathbb{E}[x_i x_{i+1}^\top])$.
2. The transition term depends on the drift only through three per-source moments under $q(x_i) = \mathcal{N}(m_i, S_i)$:

$$\text{Ef}_i = \mathbb{E}_{q(x_i)}[\bar{f}], \quad \text{Edfdx}_i = \mathbb{E}_{q(x_i)}[J_{\bar{f}}], \quad \text{Eff}_i = \mathbb{E}_{q(x_i)}[\bar{f}^\top \Sigma^{-1} \bar{f}].$$

3. **Method.** The three moments are *per source* ($\text{Ef}_i, \text{Edfdx}_i, \text{Eff}_i$ for every i). From each we assemble the per-transition scalar `trmi` (`compute_neg_CE_single`) using the $q(x)$ statistics $(d_i, C_i, m_i, S_i, S_{i,i+1})$, then sum: $\mathcal{L}(\mu) = \sum_i \text{trm}_i$. The sum is ours — formed only to hand `jax.grad` a scalar; since `trmi` depends only on μ_i , `jax.grad`($\mathcal{L}$) once returns every per-transition (J, h, L) . No hand-derived gradients and nothing dropped — reverse mode returns the exact gradient.
4. Each moment is a Gaussian average of Fourier features, i.e. a characteristic function. With the enveloped feature

$$\begin{aligned} \psi_k(m, S) &= D_k e^{2\pi i \xi_k^\top (m - x_c)} e^{-2\pi^2 \xi_k^\top S \xi_k}, \\ \mathbb{E}_{q(x_i)}[\bar{f}_r] &= \sum_k \mu_{r,k} \psi_k, \quad \mathbb{E}_{q(x_i)}[\partial_j \bar{f}_r] = \sum_k \mu_{r,k} (2\pi i \xi_{k,j}) \psi_k, \\ \mathbb{E}_{q(x_i)}[\bar{f}^\top \bar{f}] &= \sum_\delta \rho(\delta) e^{2\pi i \delta^\top (m - x_c)} e^{-2\pi^2 \delta^\top S \delta}, \quad \rho(\delta) = \sum_r \sum_{\xi_k - \xi_{k'} = \delta} (D_k \mu_{r,k}) (D_{k'} \mu_{r,k'}). \end{aligned}$$

The two paths differ *only* in how these sums — and, under `jax.grad`, their adjoints — are evaluated. $N = K(T-1)$ is the source count.

2 Path A — direct sums ($O(NM)$)

1. Increments $d_i = m_{i+1} - m_i$, $C_i = S_{i,i+1} - S_i$.
2. Autocorrelation $\rho(\delta)$: one padded FFT of $\{D_k \mu_{r,k}\}$ per output r , summed over r (once per outer iter).
3. Moments, *one per source* i , by direct summation: $\text{Ef}_i, \text{Edfdx}_i$ as the $O(M)$ spectral sums above, Eff_i as the $O(M)$ sum over the difference grid against ρ — each a value at m_i , no sum over sources. Vectorised over all N sources $\Rightarrow O(NM)$.
4. Assemble $\mathcal{L} = \sum_i \text{trm}_i$; `jax.grad`($\mathcal{L}$) w.r.t. μ . Reverse mode differentiates the dense sums directly (another $O(NM)$ pass); no NUFFT.

Cost $O(NM)$ time and memory (the reverse tape holds the (N, M) mode arrays). **Exact.** Simplest; best at small M . Implementation: `exp_full_moments.py` + `exp_batched_estep.py`.

3 Path B — gather / NUFFT ($O(M \log M + N)$)

1. Increments d_i, C_i as above.
2. Coefficient grids: $c_k^{(0)} = D_k \mu_{r,k}$ (mean), $c_{k,j}^{(1)} = (2\pi i \xi_{k,j}) D_k \mu_{r,k}$ (Jacobian); and the difference-grid $\rho(\delta)$ (one padded-FFT autocorrelation) for the quadratic.
3. Evaluate the moments by the *gmix gather* — a Gaussian-enveloped Type-2 NUFFT that reads the coefficient field at each m_i with its own $\mathcal{N}(\cdot; m_i, S_i)$ stencil, returning *one value per source* (not a sum): one batched call for Ef, one per latent dim for Edfdx, one on the ρ -grid for Eff. Cost $O(M \log M + N)$ per call. (Contrast the $q(f)$ -side *spreader*, a Type-1 NUFFT that *accumulates* $\sum_i w_i \mathcal{N}(\cdot; m_i, S_i)$ into the BTTB generator — that is the primitive that sums over sources; the gather does not.)
4. Assemble the per-source trm_i and $\mathcal{L} = \sum_i \text{trm}_i$; `jax.grad(L)`. The VJP of each gather is its Type-1 adjoint — the *spreader* (`efgp_gmix_spreader`), the same $\sum_i$ -onto-the-grid primitive used on the $q(f)$ side — chained automatically, also $O(M \log M + N)$, memory flat in M . So the whole per-source pipeline (moments $\rightarrow \text{trm}_i \rightarrow$ gradients) is $O(M \log M + N)$: the gather (Type-2) and spreader (Type-1) are an adjoint pair, forward and backward.

Cost $O(M \log M + N)$ time, memory independent of M . **Approximate:** the per-source Gaussian stencil truncates at n_σ (tunable `gather_N`, `stencil_r`); error decreases on finer grids. Best at large M/N . Primitive: `efgp_gmix_gather.py` / `drift_moments_gmix_jax`.

4 Choosing a path

	Path A (direct)	Path B (gather)
time	$O(NM)$	$O(M \log M + N)$
memory	$O(NM)$	flat in M
moments	exact	stencil-approx (n_σ)
grad	autodiff (dense)	autodiff (NUFFT adjoints)
best when	M small	M or N large

Both return the same natural gradient (to the gather’s stencil tolerance). The $O(NM)$ cost is exclusive to Path A (brute-force sums, no *gmix*); Path B’s gather/spreader pair keeps *per-source* moments and gradients at $O(M \log M + N)$. Empirically the crossover is $M \approx 200\text{--}350$: below it A is faster and exact; above it B is several-fold faster with flat memory. In neither path is any gradient hand-derived or dropped — both are `jax.grad` of the exact summed transition term; they differ only in the primitive used to evaluate (and hence adjoint) the moment sums.